

Manejo de archivos

Veremos en esta unidad cómo manejar archivos desde nuestros programas.

Existen dos formas básicas de acceder a un archivo, una es utilizarlo como un archivo de texto, que procesaremos línea por línea; la otra es tratarlo como un archivo binario, que procesaremos byte por byte.

En Python, para abrir un archivo usaremos la función `open`, que recibe el nombre del archivo a abrir.

```
archivo = open("archivo.txt")
```

Esta función intentará abrir el archivo con el nombre indicado. Si tiene éxito, devolverá una variable que nos permitirá manipular el archivo de diversas maneras.

La operación más sencilla a realizar sobre un archivo es leer su contenido. Para procesarlo línea por línea, es posible hacerlo de la siguiente forma:

```
línea=archivo.readline()
while línea != '':
    # procesar línea
    línea=archivo.readline()
```

Esto funciona ya que cada archivo que se encuentre abierto tiene una posición asociada, que indica el último punto que fue leído. Cada vez que se lee una línea, avanza esa posición. Es por ello que `readline()` devuelve cada vez una línea distinta y no siempre la misma.

La siguiente estructura es una forma equivalente a la vista en el ejemplo anterior.

```
for línea in archivo:
    # procesar línea
```

De esta manera, la variable `línea` irá almacenando distintas cadenas correspondientes a cada una de las líneas del archivo.

Es posible, además, obtener todas las líneas del archivo utilizando una sola llamada a función:

```
líneas = archivo.readlines()
```

En este caso, la variable `líneas` tendrá una lista de cadenas con todas las líneas del archivo.

Cerrar un archivo

Al terminar de trabajar con un archivo, es recomendable cerrarlo, por diversos motivos: en algunos sistemas los archivos sólo pueden ser abiertos de a un programa por la vez; en otros, lo que se

haya escrito no se guardará realmente hasta no cerrar el archivo; o el límite de cantidad de archivos que puede manejar un programa puede ser bajo, etc.

Para cerrar un archivo simplemente se debe llamar a:

```
archivo.close()
```

Advertencia Es importante tener en cuenta que cuando se utilizan funciones como `archivo.readlines()`, se está cargando en memoria el archivo completo. Siempre que una instrucción cargue un archivo completo en memoria debe tenerse cuidado de utilizarla sólo con archivos pequeños, ya que de otro modo podría agotarse la memoria de la computadora.

Ejemplo de procesamiento de archivos

Por ejemplo, para mostrar todas las líneas de un archivo, precedidas por el número de línea, podemos hacer:

Imprime las líneas de un archivo con su número

```
archivo = open("archivo.txt")
i = 1
for linea in archivo:
    linea = linea.rstrip("\n")
    print " %4d: %s" % (i, linea)
    i+=1
archivo.close()
```

La llamada a `rstrip` es necesaria ya que cada línea que se lee del archivo contiene un fin de línea y con la llamada a `rstrip("\n")` se remueve.

Nota Los archivos de texto son sencillos de manejar, pero existen por lo menos tres formas distintas de marcar un fin de línea. En Unix tradicionalmente se usa el carácter `\n` (valor de ASCII 10, definido como nueva línea) para el fin de línea, mientras que en Macintosh el fin de línea se solía representar como un `\r` (valor ASCII 13, definido como retorno de carro) y en Windows se usan ambos caracteres `\r\n`.

Si bien esto es algo que hay que tener en cuenta en una diversidad de casos, en particular en Python por omisión se maneja cualquier tipo de fin de línea como si fuese un `\n`, salvo que se le pida lo contrario. Para manejar los caracteres de fin de línea *a mano* se puede poner una `u` en el parámetro modo que le pasamos a `open`.

Otra opción para hacer exactamente lo mismo sería utilizar la función de Python `enumerate(secuencia)`. Esta función devuelve un contador por cada uno de los elementos que se recorren, puede usarse con cualquier tipo de secuencia, incluyendo archivos. La versión equivalente se muestra:

#: Imprime las líneas de un archivo con su número

```
archivo = open("archivo.txt")
for i, linea in enumerate(archivo):
    linea = linea.rstrip("\n")
    print " %4d: %s" % (i, linea)
```

```
archivo.close()
```

Modo de apertura de los archivos

La función `open` recibe un parámetro opcional para indicar el modo en que se abrirá el archivo. Los tres modos de apertura que se pueden especificar son:

- Modo de **sólo lectura** (`r`). En este caso no es posible realizar modificaciones sobre el archivo, solamente leer su contenido.
- Modo de **sólo escritura** (`w`). En este caso el archivo es truncado (vaciado) si existe, y se lo crea si no existe.
- Modo **sólo escritura posicionándose al final del archivo** (`a`). En este caso se crea el archivo, si no existe, pero en caso de que exista se posiciona al final, manteniendo el contenido original.

Por otro lado, en cualquiera de estos modos se puede agregar un `+` para pasar a un modo lectura-escritura. El comportamiento de `r+` y de `w+` no es el mismo, ya que en el primer caso se tiene el archivo completo, y en el segundo caso se trunca el archivo, perdiendo así los datos.

Nota Si un archivo no existe y se lo intenta abrir en modo lectura, se generará un error; en cambio si se lo abre para escritura, Python se encargará de crear el archivo al momento de abrirlo, ya sea con `w`, `a`, `w+` o con `a+`.

En caso de que no se especifique el modo, los archivos serán abiertos en modo sólo lectura (`r`).

Advertencia Si un archivo existente se abre en modo escritura (`w` o `w+`), todos los datos anteriores son borrados y reemplazados por lo que se escriba en él.

Escribir en un archivo

De la misma forma que para la lectura, existen dos formas distintas de escribir a un archivo.

Mediante cadenas:

```
archivo.write(cadena)
```

O mediante listas de cadenas:

```
archivo.writelines(lista_de_cadenas)
```

Así como la función `read` devuelve las líneas con los caracteres de fin de línea (`\n`), será necesario agregar los caracteres de fin de línea a las cadenas que se vayan a escribir en el archivo.

#genera_saludo.py: Genera el archivo saludo.py

```
saludo = open("[*saludo.py*](http://saludo.py)", "w")
saludo.write("""
print "Hola Mundo"
""")
saludo.close()
```

El ejemplo que se muestra contiene un programa Python que a su vez genera el código de otro programa Python.

Advertencia Si un archivo existente se abre en modo lectura-escritura, al escribir en él se sobrescribirán los datos anteriores, a menos que se haya llegado al final del archivo. Este proceso de sobrescritura se realiza carácter por carácter, sin consideraciones adicionales para los caracteres de fin de línea ni otros caracteres especiales.

Agregar información a un archivo

Abrir un archivo en modo *agregar al final* puede parecer raro, pero es bastante útil.

Uno de sus usos es para escribir un archivo de bitácora (o archivo de *log*), que nos permita ver los distintos eventos que se fueron sucediendo, y así encontrar la secuencia de pasos (no siempre evidente) que hace nuestro programa.

Esta es una forma muy habitual de buscar problemas o hacer un seguimiento de los sucesos. Para los administradores de sistemas es una herramienta esencial de trabajo.

A continuación se muestra un módulo para manejo de logs, que se encarga de la apertura del archivo, del guardado de las líneas una por una y del cerrado final del archivo.

En este módulo se utiliza el módulo de Python `datetime` para obtener la fecha y hora actual que se guardará en los archivos. Es importante notar que en el módulo mostrado no se abre o cierra un archivo en particular, sino que las funciones están programadas de modo tal que puedan ser utilizadas desde otro programa.

Se trata de un módulo genérico que podrá ser utilizado por diversos programas, que requieran la funcionalidad de registrar los posibles errores o eventos que se produzcan durante la ejecución.

Para utilizar este módulo, será necesario primero llamar a `abrir_log` para abrir el archivo de log, luego llamar a `guardar_log` por cada mensaje que se quiera registrar, y finalmente llamar a `cerrar_log` cuando se quiera concluir la registración de mensajes.

Luego se muestra un posible programa que utiliza el módulo de log incluido anteriormente.

```
# Log.py: Módulo para manipulación de archivos de Log
#! /usr/bin/env python
# encoding: latin1

# El módulo datetime se utiliza para obtener la fecha y hora actual.
import datetime

def abrir_log(nombre_log):
    """ Abre el archivo de log indicado. Devuelve el archivo abierto.
    Pre: el nombre corresponde a un nombre de archivo válido.
    Post: el archivo ha sido abierto posicionándose al final. """
```

```

    archivo_log = open(nombre_log, "a")
    guardar_log(archivo_log, "Iniciando registro de errores")
    return archivo_log

def guardar_log(archivo_log, mensaje):
    """ Guarda el mensaje en el archivo de log, con la hora actual.
    Pre: el archivo de log ha sido abierto correctamente.
    Post: el mensaje ha sido escrito al final del archivo. """
    # Obtiene la hora actual en formato de texto
    hora_actual = str(datetime.datetime.now())
    # Guarda la hora actual y el mensaje de error en el archivo
    archivo_log.write(hora_actual+" "+mensaje+"\n")

def cerrar_log(archivo_log):
    """ Cierra el archivo de log.
    Pre: el archivo de log ha sido abierto correctamente.
    Post: el archivo de log se ha cerrado. """
    guardar_log(archivo_log, "Fin del registro de errores")
    archivo_log.close()

# usa_log.py: Módulo que utiliza el módulo de log
#!/usr/bin/env python
#encoding latin1

import log

# Constante que contiene el nombre del archivo de log a utilizar
ARCHIVO_LOG = "mi_log.txt"

def main():
    # Al comenzar, abrir el log
    archivo_log = log.abrir_log(ARCHIVO_LOG)
    # ...
    # Hacer cosas que pueden dar error
    if error:
        guardar_log(archivo_log, "ERROR: "+error)
    # ...
    # Finalmente cerrar el archivo
    log.cerrar_log(archivo_log)

main()

```

Este código, que incluye el módulo `log` mostrado anteriormente, muestra una forma básica de utilizar un archivo de log. Al iniciarse el programa se abre el archivo de log, de forma que queda registrada la fecha y hora de inicio. Posteriormente se realizan tareas varias que podrían provocar errores, y de haber algún error se lo guarda en el archivo de log. Finalmente, al terminar el programa, se cierra el archivo de log, quedando registrada la fecha y hora de finalización.

El archivo de log generado tendrá la forma:

```

2015-04-10 15:20:32.229556 Iniciando registro de errores
2015-04-10 15:20:50.721415 ERROR: no se pudo acceder al recurso
2015-04-10 15:21:58.625432 ERROR: formato de entrada inválido
2015-04-10 15:22:10.109376 Fin del registro de errores

```

Manipular un archivo en forma binaria

No todos los archivos son archivos de texto, y por lo tanto no todos los archivos pueden ser procesados por líneas. Existen archivos en los que cada byte tiene un significado particular, y es necesario manipularlos conociendo el formato en que están los datos para poder procesar esa información.

Para abrir un archivo y manejarlo de forma binaria es necesario agregarle una `b` al parámetro de modo.

Para procesar el archivo de a bytes en lugar de líneas, se utiliza la función `contenido = archivo.read(n)` para leer `n` bytes y `archivo.write(contenido)`, para escribir contenido en la posición actual del archivo.

Nota La `b` en el modo de apertura viene de *binario*, por el sistema de numeración binaria, ya que en el procesador de la computadora la información es manejada únicamente mediante ceros o unos (bits) que conforman números binarios.

Si bien no es necesaria en todos los sistemas (en general el mismo sistema detecta que es un archivo binario sin que se lo pidamos), es una buena costumbre usarla, por más que sirva principalmente como documentación.

Al manejar un archivo binario, es necesario poder conocer la posición actual en el archivo y poder modificarla. Para obtener la posición actual se utiliza `archivo.tell()`, que indica la cantidad de bytes desde el comienzo del archivo.

Para modificar la posición actual se utiliza `archivo.seek(inicio, desde)`, que permite desplazarse una cantidad `inicio` de bytes en el archivo, contando desde el comienzo del archivo, desde la posición actual o desde el final.

Persistencia de datos

Se llama **persistencia** a la capacidad de guardar la información de un programa para poder volver a utilizarla en otro momento. Es lo que los usuarios conocen como *Guardar el archivo* y después *Abrir el archivo*. Pero para un programador puede significar más cosas y suele involucrar un proceso de *serialización* de los datos a un archivo o a una base de datos o a algún otro medio similar, y el proceso inverso de recuperar los datos a partir de la información *serializada*.

Por ejemplo, supongamos que en el desarrollo de un juego se quiere guardar en un archivo la información referente a los ganadores, el puntaje máximo obtenido y el tiempo de juego en el que obtuvieron ese puntaje.

En el juego, esa información podría estar almacenada en una lista de tuplas:

```
[(nombre1, puntaje1, tiempo1), (nombre2, puntaje2, tiempo2), ...]
```

Esta información se puede guardar en un archivo de muchas formas distintas. En este caso, para facilitar la lectura del archivo de puntajes para los humanos, se decide guardarlos en un archivo de texto, donde cada tupla ocupará una línea y los valores de las tuplas estarán separados por comas.

En el código siguiente se muestra un módulo capaz de guardar y recuperar los puntajes en el formato especificado.

Dadas las especificaciones del problema al guardar los valores en el archivo, es necesario convertir el puntaje (que es un valor numérico) en una cadena, y al abrir el archivo es necesario convertirlo nuevamente en un valor numérico.

Nota Es importante notar que tanto la función que almacena los datos como la que los recupera requieren que la información se encuentre de una forma determinada y de no ser así, fallarán. Es por eso que estas condiciones se indican en la documentación de las funciones como sus precondiciones. En próximas unidades veremos cómo evitar que falle una función si alguna de sus condiciones no se cumple.

Es bastante sencillo probar el módulo programado y ver que lo que se guarda es igual que lo que se recupera:

puntajes.py: Módulo para guardar y recuperar puntajes en un archivo

```
#!/usr/bin/env python
# encoding: latin1

def guardar_puntajes(nombre_archivo, puntajes):
    """ Guarda la lista de puntajes en el archivo.
    Pre: nombre_archivo corresponde a un archivo válido,
        puntajes corresponde a una lista de tuplas de 3 elementos.
    Post: se guardaron los valores en el archivo,
        separados por comas.
    """

    archivo = open(nombre_archivo, "w")
    for nombre, puntaje, tiempo in puntajes:
        archivo.write(nombre+","+str(puntaje)+","+tiempo+"\n")
    archivo.close()

def recuperar_puntajes(nombre_archivo):
    """ Recupera los puntajes a partir del archivo provisto.
    Devuelve una lista con los valores de los puntajes.
    Pre: el archivo contiene los puntajes en el formato esperado,
        separados por comas
    Post: la lista devuelta contiene los puntajes en el formato:
        [(nombre1,puntaje1,tiempo1),(nombre2,puntaje2,tiempo2)].
    """

    puntajes = []
    archivo = open(nombre_archivo, "r")
    for línea in archivo:
        nombre, puntaje, tiempo = línea.rstrip("\n").split(",")
        puntajes.append((nombre,int(puntaje),tiempo))
    archivo.close()
    return puntajes

>>> import puntajes
>>> valores = [("Pepe", 108, "4:16"), ("Juana", 2315, "8:42")]
```

```
>>> puntajes.guardar_puntajes("puntajes.txt", valores)
>>> recuperado = puntajes.recuperar_puntajes("puntajes.txt")
>>> print recuperado
[('Pepe', 108, '4:16'), ('Juana', 2315, '8:42')]
```

Guardar el estado de un programa se puede hacer tanto en un archivo de texto, como en un archivo binario. En muchas situaciones es preferible guardar la información en un archivo de texto, ya que de esta manera es posible modificarlo fácilmente desde cualquier editor de textos.

En general, los archivos de texto van a desperdiciar un poco más de espacio, pero son más fáciles de entender y fáciles de usar desde cualquier programa.

Por otro lado, en un archivo binario bien definido se puede evitar el desperdicio de espacio, o también hacer que sea más rápido acceder a los datos. Además, para ciertas aplicaciones como archivos de sonido o video, tendría poco sentido almacenarlos en archivos de texto.

En definitiva, la decisión de qué formato usar queda a discreción del programador. Es importante recordar que el sentido común es el valor máspreciado en un programador.

Persistencia en archivos CSV

Un formato que suele usarse para transferir datos entre programas es **CSV** (del inglés *comma separated values*: valores separados por comas) es un formato bastante sencillo, tanto para leerlo como para procesarlo desde el código, se parece al formato visto en el ejemplo anteriormente.

```
Nombre,Apellido,Telefono,Cumpleaños
"John","Smith","555-0101","1973-11-24"
"Jane","Smith","555-0101","1975-06-12"
```

En el ejemplo se puede ver una pequeña base de datos. En la primera línea del archivo tenemos los nombres de los campos, un dato opcional desde el punto de vista del procesamiento de la información, pero que facilita entender el archivo.

En las siguientes líneas se ingresan los datos de la base de datos, cada campo separado por comas. Los campos que son cadenas se suelen escribir entre comillas dobles, si alguna cadena contiene alguna comilla doble se la reemplaza por \" y una contrabarra se escribe como \\\.

En Python es bastante sencillo procesar de este tipo de archivos, tanto para la lectura como para la escritura, mediante el módulo `csv` que ya se encuentra preparado para eso.

Las funciones del ejemplo anterior podría programarse mediante el módulo `csv`. En el Código se muestra una posible implementación que utiliza este módulo.

Si se prueba este código, se obtiene un resultado idéntico al obtenido anteriormente:

```
>>> import puntajes_csv
>>> valores = [("Pepe", 108, "4:16"), ("Juana", 2315, "8:42")]
>>> puntajes_csv.guardar_puntajes("puntajes.txt", valores)
>>> recuperado = puntajes_csv.recuperar_puntajes("puntajes.txt")
>>> print recuperado
```



```
[('Pepe', 108, '4:16'), ('Juana', 2315, '8:42')]
```

El código, en este caso, es muy similar, ya que en el ejemplo original se hacían muy pocas consideraciones al respecto de los valores: se asumía que el primero y el tercero eran cadenas mientras que el segundo necesitaba ser convertido a cadena.

puntajes_csv.py: Módulo para guardar y recuperar puntajes en un archivo que usa csv

```
#!/usr/bin/env python
# encoding: Latin1

import csv

def guardar_puntajes(nombre_archivo, puntajes):
    """ Guarda la lista de puntajes en el archivo.
    Pre: nombre_archivo corresponde a un archivo válido,
        puntajes corresponde a una lista de secuencias de elementos.
    Post: se guardaron los valores en el archivo,
        separados por comas.
    """

    archivo = open(nombre_archivo, "w")
    archivo_csv = csv.writer(archivo)
    archivo_csv.writerows(puntajes)
    archivo.close()

def recuperar_puntajes(nombre_archivo):
    """ Recupera los puntajes a partir del archivo provisto.
    Devuelve una lista con los valores de los puntajes.
    Pre: el archivo contiene los puntajes en el formato esperado,
        separados por comas
    Post: la lista devuelta contiene los puntajes en el formato:
        [(nombre1,puntaje1,tiempo1),(nombre2,puntaje2,tiempo2)].
    """

    puntajes = []
    archivo = open(nombre_archivo, "r")
    archivo_csv = csv.reader(archivo)
    for nombre, puntaje, tiempo in archivo_csv:
        puntajes.append((nombre, int(puntaje), tiempo))
    archivo.close()
    return puntajes
```

Nota Es importante notar, entonces, que al utilizar el módulo csv en lugar de hacer el procesamiento en forma manual, se obtiene un comportamiento más robusto, ya que el módulo csv tiene en cuenta muchos más casos que nuestro código original no. Por ejemplo, el código anterior no tenía en cuenta que el nombre pudiera contener una coma.

En el apéndice de esta unidad puede verse una aplicación completa de una agenda, que almacena los datos del programa en archivos csv.

Persistencia en archivos binarios

En el caso de que decidiéramos grabar los datos en un archivo binario, Python incluye una herramienta llamada **pickle** que permite hacerlo de forma muy sencilla. Hay que tener en cuenta, sin embargo, que no es nada simple acceder a un archivo en este formato desde un programa que no esté escrito en Python.

Se muestra el mismo ejemplo de almacenamiento de puntajes, utilizando el módulo `pickle`.

puntajes_pickle.py: Módulo para guardar y recuperar puntajes en un archivo que usa pickle

```
#!/usr/bin/env python
# encoding: Latin1

import pickle

def guardar_puntajes(nombre_archivo, puntajes):
    """ Guarda la lista de puntajes en el archivo.
    Pre: nombre_archivo corresponde a un archivo válido,
        puntajes corresponde a los valores a guardar
    Post: se guardaron los valores en el archivo en formato pickle.
    """

    archivo = open(nombre_archivo, "w")
    pickle.dump(puntajes, archivo)
    archivo.close()

def recuperar_puntajes(nombre_archivo):
    """ Recupera los puntajes a partir del archivo provisto.
    Devuelve una lista con los valores de los puntajes.
    Pre: el archivo contiene los puntajes en formato pickle
    Post: la lista devuelta contiene los puntajes en el
        mismo formato que se los almacenó.
    """

    archivo = open(nombre_archivo, "r")
    puntajes = pickle.load(archivo)
    archivo.close()
    return puntajes
```

El funcionamiento de este programa será idéntico a los anteriores. Pero el archivo generado será muy distinto a los archivos generados anteriormente. En lugar de ser un archivo de fácil lectura, tendrá la forma:

```
(lp0
(S'Pepe'
p1
I108
S'4:16'
p2
tp3
a(S'Juana'
p4
I2315
S'8:42'
p5
tp6
```

En el apéndice puede verse una aplicación completa de una agenda, que utiliza pickle para almacenar datos en archivos.

Directorios

Hasta aquí se ha mostrado el acceso a los archivos utilizando sólo el nombre del archivo, esto nos permite acceder a los archivos en el directorio actual donde corre el programa.

Un problema relacionado con la utilización de directorios es que los separadores de directorios en distintos sistemas son distintos, / en Unix y Macintosh, \ en Windows. La manera de acceder a directorios independientemente del sistema en el que estamos desde Python es usando el modulo `os`.

```
os.path.join("data", "archivo.csv")
```

Apéndice

A continuación, el código para un programa de agenda que utiliza archivos csv. Luego, los cambios necesarios para que la agenda que utilice archivos en formato pickle, en lugar de csv.

```
# agenda-csv.py: Agenda con los datos en csv

#!/usr/bin/env python
# encoding: Latin1

import csv

ARCHIVO="agenda.csv"
CAMPOS=["Nombre", "Apellido", "Telefono", "Cumpleaños"]

def leer_csv(datos_csv):
    """ Devuelve la siguiente línea o None si se terminó el archivo. """
    try:
        return datos_csv.next()
    except:
        return None

def leer_datos(archivo):
    """ Carga todos los datos del archivo en una lista y la devuelve. """
    abierto = open(archivo)
    datos_csv = csv.reader(abierto)
    campos = leer_csv(datos_csv)
    datos = []
    elemento = leer_csv(datos_csv)
    while elemento:
        datos.append(elemento)
        elemento = leer_csv(datos_csv)
    abierto.close()
    return datos
```

```

def guardar_datos(datos, archivo):
    """ Guarda los datos recibidos en el archivo. """
    abierto = open(archivo,"w")
    datos_csv = csv.writer(abierto)
    datos_csv.writerow(CAMPOS)
    datos_csv.writerows(datos)
    abierto.close()

def leer_búsqueda():
    """ Solicita al usuario nombre y apellido y los devuelve. """
    nombre = raw_input("Nombre: ")
    apellido = raw_input("Apellido: ")
    return (nombre,apellido)

def buscar(nombre, apellido, datos):
    """ Busca el primer elemento que coincida con nombre y con apellido. """
    for elemento in datos:
        if nombre in elemento[0] and apellido in elemento[1]:
            return elemento
    return None

def menu_alta(nombre, apellido, datos):
    """ Pregunta si se desea ingresar un nombre y apellido y
        de ser así, pide los datos al usuario. """
    print "No se encuentra %s %s en la agenda." % (nombre, apellido)
    confirmacion = raw_input("¿Desea ingresarlo? (s/n) ")
    if confirmacion.lower() != "s":
        return
    telefono = raw_input("Telefono: ")
    cumple = raw_input("Cumpleaños: ")
    datos.append([nombre,apellido,telefono,cumple])

def mostrar_elemento(elemento):
    """ Muestra por pantalla un elemento en particular. """
    print
    print " %s %s" % (elemento[0],elemento[1])
    print "Telefono: %s" % elemento[2]
    print "Cumpleaños: %s" % elemento[3]
    print

def menu_elemento():
    """ Muestra por pantalla las opciones disponibles para un elemento
        existente. """
    o = raw_input("b: borrar, m: modificar, ENTER para continuar (b/m): ")
    return o.lower()

def modificar(viejo, nuevo, datos):
    """ Reemplaza el elemento viejo con el nuevo, en la lista datos. """
    indice = datos.index(viejo)
    datos[indice] = nuevo

def menu_modificacion(elemento, datos):
    """ Solicita al usuario los datos para modificar una entrada. """
    nombre = raw_input("Nuevo nombre: ")
    apellido = raw_input("Nuevo apellido: ")
    telefono = raw_input("Nuevo teléfono: ")
    cumple = raw_input("Nuevo cumpleaños: ")
    modificar(elemento, [nombre, apellido, telefono, cumple], datos)

def baja(elemento, datos):
    """ Elimina un elemento de la lista. """

```

```

    datos.remove(elemento)

def confirmar_salida():
    """ Solicita confirmación para salir """
    confirmacion = raw_input("¿Desea salir? (s/n) ")
    return confirmacion.lower() == "s"

def agenda():
    """ Función principal de la agenda.
        Carga los datos del archivo, permite hacer búsquedas, modificar
        borrar, y al salir guarda. """
    datos = leer_datos(ARCHIVO)
    fin = False
    while not fin:
        (nombre, apellido) = leer_busqueda()
        if nombre == "" and apellido == "":
            fin = confirmar_salida()
            continue
        elemento = buscar(nombre, apellido, datos)
        if not elemento:
            menu_alta(nombre, apellido, datos)
            continue
        mostrar_elemento(elemento)
        opcion = menu_elemento()
        if opcion == "m":
            menu_modificacion(elemento, datos)
        elif opcion == "b":
            baja(elemento, datos)
        guardar_datos(datos, ARCHIVO)

agenda()
# agenda-pickle.py: Diferencia de agenda con datos en pickle

import pickle

ARCHIVO="agenda.dat"

def leer_datos(archivo):
    """ Carga todos los datos del archivo en una lista y la devuelve."""
    abierto = open(archivo)
    datos = pickle.load(archivo)
    abierto.close()
    return datos

def guardar_datos(datos, archivo):
    """ Guarda los datos recibidos en el archivo. """
    abierto = open(archivo,"w")
    pickle.dump(archivo, datos)
    abierto.close()

```